Processors

M. Sonza Reorda

Politecnico di Torino
Dip. di Automatica e Informatica



Introduction

A processor may correspond to

- **A module (*core*) within an IC**
- **A device by itself.**

Different types of processors exist, which differ for

- **Complexity**
- **Computational power**
- **Architecture**
- **…**

Introduction

A processor performs two types of operations:

- **It executes instructions (whose code is in memory)**
- **It interacts with the outside world (through appropriate interfaces).**

3

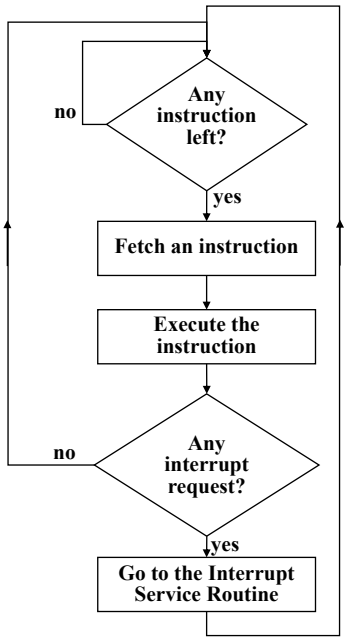
External events

The processor interacts with the outside world in different manners

- **Via the system bus**
  - **To access the memory modules**
  - **To access peripherals**
- **Via the interrupt mechanism.**

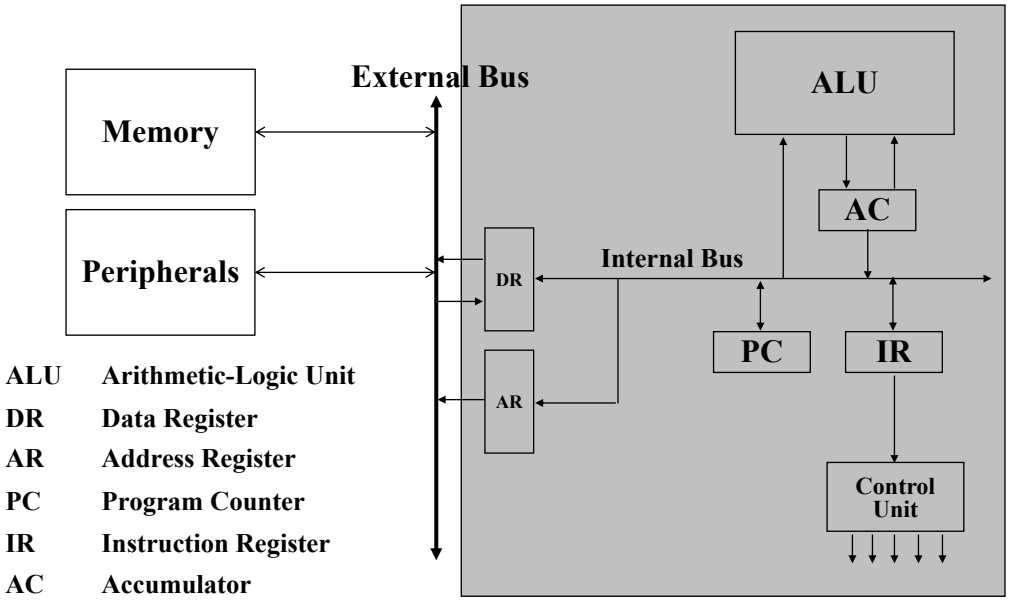
4

Processor behavior



5

CPU minimal architecture



ALU	Arithmetic-Logic Unit
DR	Data Register
AR	Address Register
PC	Program Counter
IR	Instruction Register
AC	Accumulator

6

Instructions

The execution of each machine instruction consists of two phases:

- **fetch:**
  - **The instruction code is read from the memory**
- **execute:**
  - **the code is first decoded, then executed. This usually involves accessing one or more operands, performing an operation on them, writing the result.**

The combination of the two phases is called *instruction cycle*.

Registers

The time to access the memory is usually higher than the time it takes for the CPU to process the data.

Memory access is therefore a bottleneck for CPU performance.

For this reason, there are some very fast memory cells, known as *registers*, inside the CPU.

Where possible, operations are performed using registers to store operands and results.

Example

All Intel processors (starting from 8086) contain the following registers (initially composed of 16 bits, then extended to 32 bits):

- **AX, BX, CX, DX**
- **SI, DI**
- **DS, ES, CS, SS**
- **SP, BP.**

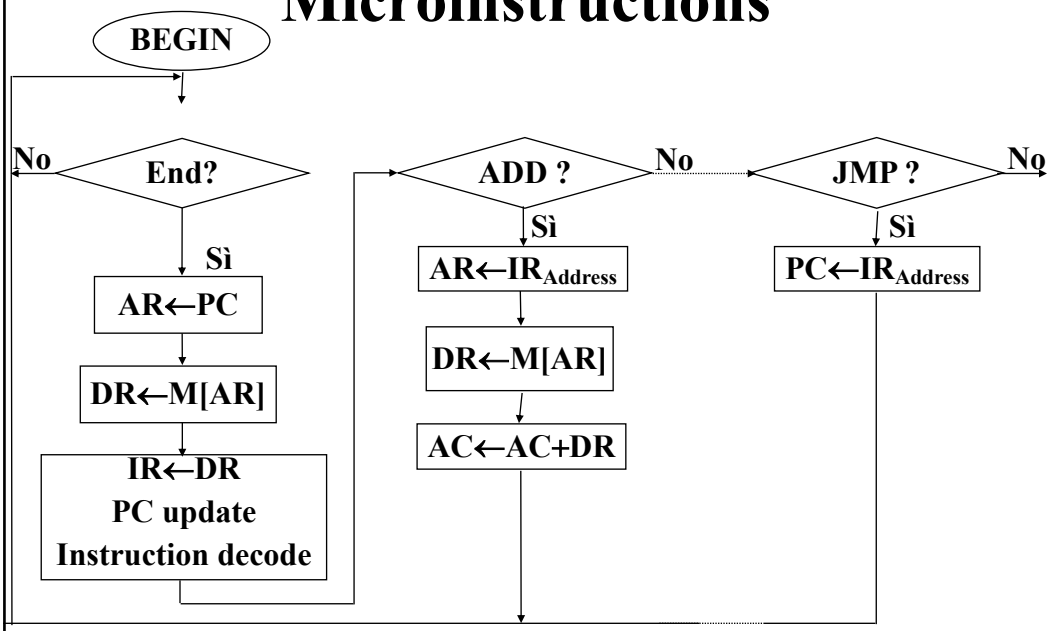
Microinstructions

Executing the two phases of an instruction corresponds to the activation of a number of *microinstructions* (e.g., for transferring a value from one register to another).

The time to run a microinstruction is defined as *CPU cycle time* and often coincides with the processor clock cycle.

In traditional processors, each instruction therefore requires a time equal to a number of clock periods.

Microinstructions



11

11

Memory access

It is performed in two phases:

1. **The processor writes**
   - **the address of the memory cell on the bus**
   - **the proper control signals on the control bus**
2. **The memory places the content of the cell on the data bus (*read*), or**
3. **The processor places on the data bus the data to be written in the addressed cell, which is acquired from the memory (*write*).**

12

12

Input/Output

Access to I / O devices is performed in a manner similar to what is used for memory access.

Each I / O device is associated with an interface device that contains a number of registers (also called *ports*) visible to the CPU through the bus.

Each interface register corresponds to an address on which the CPU can perform read or write operations.

Example

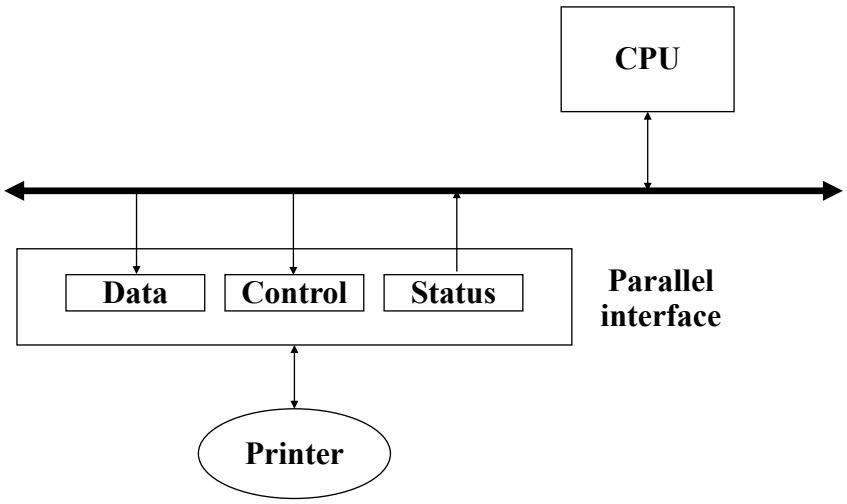
A printer is normally connected to the processing system through a parallel interface.

The printer will be viewed by the CPU through 3 registers corresponding to as many addresses as:

- **a data register (output)**
- **a control register for the control information (output)**
- **a status register for status information (input).**

To perform operations on the printer, the CPU will read or write from/to the 3 registers.

Example



Enhancements

Compared to the basic scheme of a processor, some enhancements are normally adopted for the following points:

- **registers**
- **arithmetic units**
- **status register**
- **stack.**

Registers

The number of registers strongly influences the performance of a processor.

Some registers may be restricted to some special use:

- **Data registers**
- **Index registers**
- **Counter registers.**

Arithmetic units

They can be more or less powerful, depending on the operations they implement:

- **sum and subtraction in fixed-point**
- **multiplication and division in fixed-point**
- **floating-point operations.**

Status register

It is a special register containing two groups of bits (or *flags*)

- **Status bits**
- **Control bits.**

Status bits

The status bits are

- **Written by the occurrence of particular conditions (carry, overflow, positive or negative result, null result, equality) following the execution of the instructions**
- **tested by conditional jump instructions (and some others).**

In this way, the assembly code implements the conditioned jumps.

Example

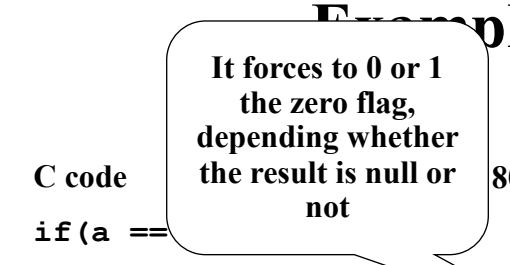
C code

```
if(a == b)
      a = 1;
```

80x86 assembly code

```
        MOV   AX, a
        MOV   BX, b
        SUB   AX, BX
        JNZ   lab
        MOV   AX, 1
lab:    MOV   a, AX
        ...
```


Example

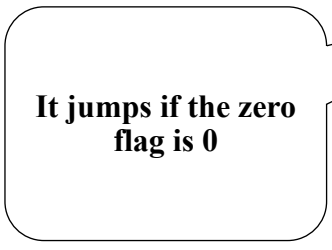


C code

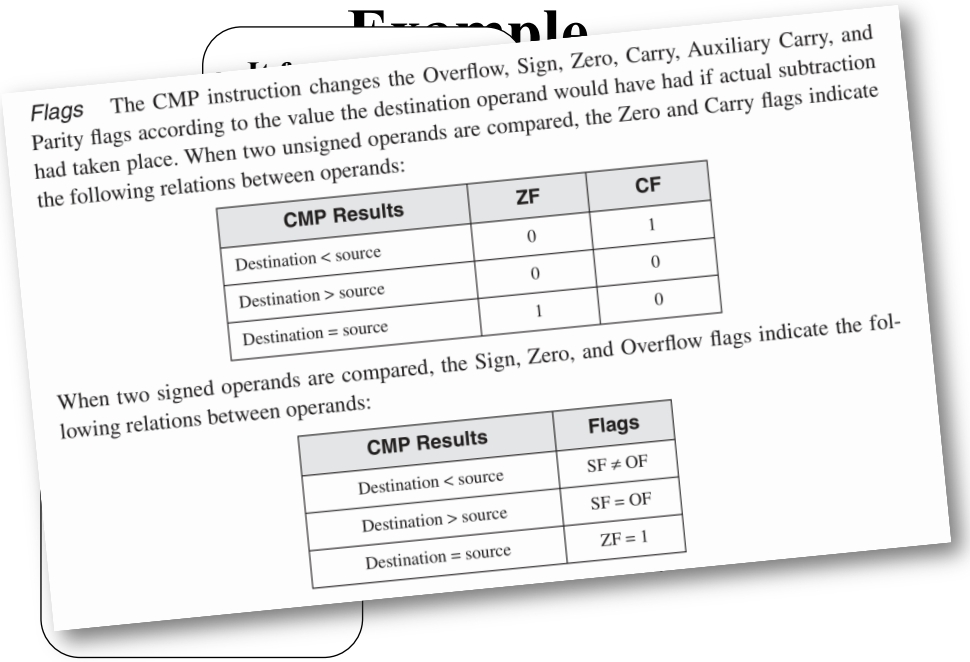
```
if(a ==
      a = 1;
```

80x86 assembly code

```
        MOV   AX, a
        MOV   BX, b
        SUB   AX, BX
        JNZ   lab
        MOV   AX, 1
lab:    MOV   a, AX
        ...
```



Example



Flags The CMP instruction changes the Overflow, Sign, Zero, Carry, Auxiliary Carry, and Parity flags according to the value the destination operand would have had if actual subtraction had taken place. When two unsigned operands are compared, the Zero and Carry flags indicate the following relations between operands:

CMP Results	ZF	CF
Destination < source	0	1
Destination > source	0	0
Destination = source	1	0

When two signed operands are compared, the Sign, Zero, and Overflow flags indicate the following relations between operands:

CMP Results	Flags
Destination < source	SF ≠ OF
Destination > source	SF = OF
Destination = source	ZF = 1

Control bits

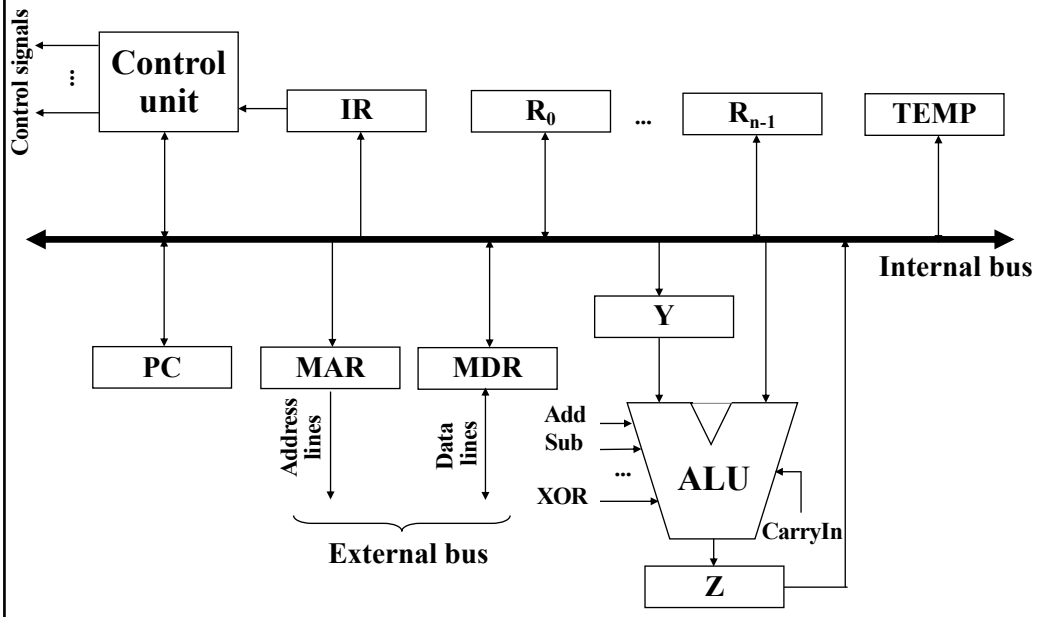
They decide the behavior of the processor with respect to a certain aspect.

For example

- **the interrupt bit controls the interrupt sensitivity**
- **the trap bit runs or does not run the processor in debug mode.**

These bits are forced to 0 or 1 by proper instructions.

CPU architecture



Stack

It is a LIFO structure existing in the memory.

Many processors have:

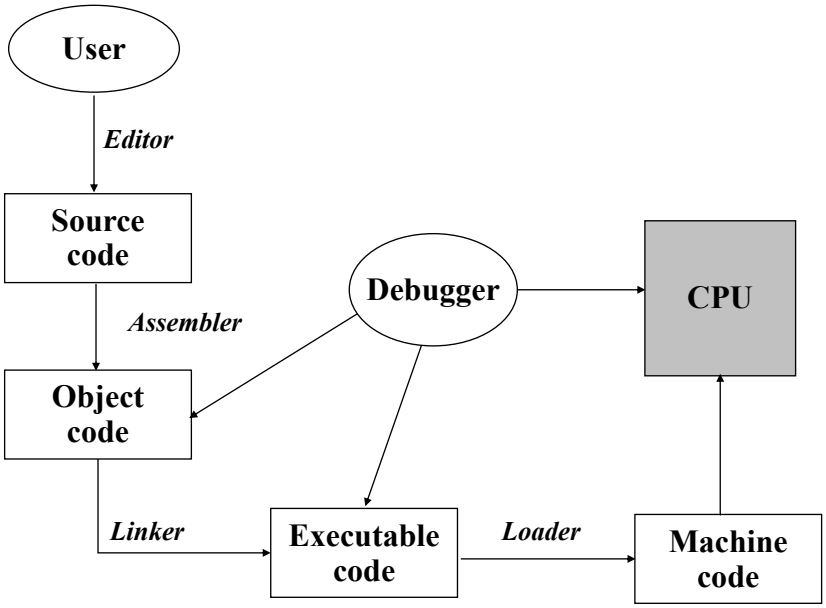
- **a special-purpose register called *Stack Pointer* (SP) pointing to the top element**
- **two instructions (*PUSH* and *POP*) that implicitly manipulate SP.**

Stack: usage

The stack is used:

- **for procedure calls (normal or interrupt service)**
- **for passing parameters to the procedures**
- **for storing local variables.**

Program life cycle



Assembler

The source file is transformed into object files by:

- **Removing comments**
- **associating each symbolic variable to an appropriate memory location**
- **machine translation of each instruction.**

Some operations may not yet be completed, such as those that concern:

- **the variables defined in other modules**
- **the procedures defined in other modules.**

29

Linker

You can create the executable file from multiple object files.

The linker verifies the correctness of calls to variables and procedures defined in other modules and generates the appropriate addresses.

The product file is not usually instantly executable since

- **it is stored on disk instead of in memory**
- **It must be independent of the memory location in which it will be loaded.**

30

Loader

It is part of the Operating System.

When invoked

- **it finds the executable file on the disk**
- **it loads it in memory, making any changes made necessary after its memory location has been determined**
- **it forces the processor to start running the program.**

Debugger

It interacts with assembly, link and execution processes, allowing the programmer to have functionalities such as:

- **breakpoints**
- **step-by-step execution**
- **access to variables and registers (in read and write)**
- **...**

Assembly languages

Each processor has its own assembly language defined through the set of assembler instructions that it can execute.

For each instruction the processor manufacturer provides

- **the operations performed**
- **the corresponding machine code.**

Different processors belonging to the same *family* can accept the same machine code.

Characteristics of an Assembly language

- **Supported operations**
- **Types of data**
- **Instructions Format**
- **Usable registers**
- **Addressing Modes**
- **Ease of use**
- **...**

Addressing modes

They correspond to the different ways in which operands can be specified:

- **Immediate Operands**
- **Registers**
- **Memory cells.**

Instruction format

At the machine code level, the instructions are formed by 2 parts:

- **the *operating code* of the instruction (opcode)**
- **the operand information.**

opcode	operand 1	operand 2